

PRÁCTICA 2: CRIPTOGRAFÍA

OBJETIVOS GENERALES

- Comprender los fundamentos matemáticos de las aplicaciones de la teoría de números a la criptografía. En particular, comprender las bases teóricas del sistema RSA.
- Construir una librería “rsa.py” que, utilizando la librería “modular.py” desarrollada en la práctica anterior, permita establecer comunicaciones encriptadas utilizando RSA.
- Usar dicha librería para construir un chat cifrado extremo a extremo “criptochat.py”, junto con un programa auxiliar “registrarusuario.py” que permita crear nuevos usuarios.

DESCRIPCIÓN FUNCIONAL DETALLADA DEL PROYECTO

El producto final que se desarrollará con el segundo proyecto es un chat con encriptación extremo a extremo. Para usar el sistema, existirá un script que permitirá dar de alta nuevos usuarios. Al dar de alta a un usuario en el cliente de chat, se le asignará un par de claves para RSA, que, por simplicidad, se almacenarán en ficheros en una carpeta “Usuarios” (en un entorno realista nunca se almacenaría una clave privada desprotegida, pero, por motivos didácticos, y con el fin de comprender bien el funcionamiento interno de RSA paso a paso, en esta práctica guardaremos las claves públicas y privadas en ficheros de texto sin cifrados adicionales). En esta misma carpeta, se copiarían también los ficheros con las claves públicas de cualquier otro usuario externo con el que se desee establecer una comunicación.

Dados dos usuarios (uno registrado en la aplicación, del que se tenga guardada su clave privada, y otro, posiblemente externo, del que se tenga guardada su clave pública), la aplicación permitirá mantener una conversación cifrada entre ambos.

Para ello, se implementarán como mínimo tres scripts, sin perjuicio de que cada grupo de prácticas quiera añadir cualquier módulo adicional que necesite para implementar sus algoritmos:

- **“rsa.py”**: Librería con la funcionalidad básica de un sistema RSA. Contendrá las funciones necesarias para crear claves RSA y utilizarlas para cifrar y descifrar mensajes (tanto numéricos como de texto) usando un esquema de padding básico. También contendrá funciones que permitan realizar ataques básicos sobre un sistema RSA pequeño o sin padding.

La lista de funciones que deben implementarse en la librería está especificada más adelante en el presente documento y se proporciona con los materiales de la práctica una plantilla con el prototipo de dichas funciones y un conjunto de tests de “pytest” para probar su funcionamiento básico.

- **“registrarusuario.py”**: Programa que permite crear un nuevo usuario. El programa se ejecutará de la siguiente manera en una terminal:

```
python registrarusuario.py
```

El programa solicitará al usuario que introduzca por teclado:

- Un nombre: <nombre>
- El valor mínimo de cada primo usado para generar sus claves de RSA.
- El valor máximo de cada primo usado para generar sus claves de RSA.
- El número de cifras de padding para la comunicación con ese usuario.

En base a estos datos, el programa generará un par de claves pública (n, e) y privada d para RSA y creará dos ficheros:

- pub_<nombre>.txt: Que contendrá 3 líneas:
 - * n
 - * e
 - * El número de cifras de padding.
- priv_<nombre>.txt: Que contendrá una única línea con la clave privada d .

y guardará estos dos ficheros en una carpeta llamada “Usuarios” ubicada en el mismo directorio que el script. Si la carpeta no existe, el programa deberá crearla. En los materiales de la práctica se incluyen ejemplos (“pub_ejemplo.txt” y “priv_ejemplo.txt”) de estos dos ficheros, para un usuario llamado “ejemplo” cuya configuración para RSA sea:

- $n = 809570247883$
- $e = 77204266853$
- $d = 42840500717$
- 3 cifras de “padding”.

- **“criptochat.py”**: Chat encriptado extremo a extremo. Se ejecutará de la siguiente manera en una terminal

python criptochat.py <usuario1> <usuario2>

donde <usuario1> y <usuario2> son los nombres de dos usuarios del programa. El programa buscará los ficheros de claves pública y privada de <usuario1> y el fichero con la clave pública de <usuario2> y cargará dichas claves. Si alguno no existe, se lo indicará al usuario y finalizará. Una vez cargadas las claves de ambos usuarios, el programa permitirá **cifrar mensajes de parte de <usuario1> para <usuario2> o descifrar mensajes que <usuario2> haya enviado a <usuario1>**. Para ello, el programa:

- Preguntará al usuario si desea cifrar (C), descifrar (D) o salir (S).
- Si el usuario escribe C: Le pedirá que escriba por teclado un texto plano e imprimirá por pantalla el correspondiente texto cifrado, usando para ello la clave pública de <usuario2> y el número de cifras de “padding” del <usuario2>.
- Si el usuario escribe D: Le pedirá que escriba por teclado el texto cifrado proveniente de <usuario2>, lo descifrá usando la clave privada de <usuario1> y el número de cifras de “padding” de <usuario1> e imprimirá por pantalla el texto claro resultante.
- Si el usuario escribe S: Saldrá del programa.

Cada vez que se realice una acción, se volverá a preguntar si desea cifrar, descifrar o salir hasta que el usuario decida salir del programa. En cuanto al formato de los mensajes:

- Los mensajes en texto plano, se introducirán e imprimirán como cadenas de texto normales.

Ejemplo de texto plano: *hola*

- Los mensajes cifrados se introducirán e imprimirán como listas de enteros separadas por espacios.

Ejemplo de texto cifrado: *512533741 128348154 125512341 16453783 3012812 1247503 50252341*

PRÁCTICA EN GRUPO: FUNDAMENTOS DE RSA

1. En la sección de prácticas de Moodle, regístrate en la mesa en la que estás sentado (Mesa A, Mesa B, Mesa C,...). Una vez hecho esto, se desbloqueará debajo un chat, en el que podrás ver una sala por cada mesa. **De momento, no escribas nada en ninguna de las salas.**

2. En cada mesa, generad un par de clave pública y clave privada para RSA. Para ello:

- Escoged dos números primos entre 800.000 y 1.000.000, p_1 y p_2 .
- Como módulo, utilizad $n = p_1 p_2$.
- Calculad $\varphi(n) = (p_1 - 1)(p_2 - 1)$.
- Como clave pública, escoged e entre 2 y $\varphi(n) - 1$ que sea coprimo con $\varphi(n)$.
- Calculad $d \equiv e^{-1} \pmod{\varphi(n)}$.

La clave pública será (n, e) y la privada d .

3. Entrad en el “chat encriptado” de Moodle y accede a la sala de tu mesa. Uno de los miembros de la mesa debe publicar el mensaje:

“Clave pública: $n\ e$ ”

Donde n y e son el módulo y exponente de la clave pública calculada en el paso anterior.

4. Escoged un número entre 1000 y 10000, m . Vamos a enviar m cifrado a la siguiente mesa (A envía a B, B envía a C,..., la última envía a A). Para ello:

- Entrad en la sala de chat del equipo al que quieres enviar el mensaje y recuperad su clave pública $(n_{destino}, e_{destino})$
- Calculad:

$$c = m^{e_{destino}} \pmod{n_{destino}}$$

- En la sala de chat del destinatario, un miembro de vuestra mesa envía el mensaje

“X: c ”

donde X es vuestra mesa y c es el número cifrado que habéis hallado. Por ejemplo, en la sala de C, el equipo B debería escribir algo similar a:

“B: 123513”

5. Entrad a vuestra sala, leed el mensaje c que os ha llegado y descifradlo. Para ello, calculad:

$$m = c^d \pmod{n}$$

donde n es vuestro módulo y d vuestra clave privada. Comprobad con vuestros compañeros de la mesa anterior si el número que habéis recuperado es el correcto.

6. Ahora vamos a cifrar un texto. Como primera aproximación (que, como veremos, no es muy segura criptográficamente), vamos a cifrar letra a letra. Para ello, colaborad en cada mesa para hacer lo siguiente:

- Escoged un mensaje (no muy largo) que le queráis enviar a la mesa anterior a la vuestra (B envía a A, C envía a B, etc. y A envía a la última).
- Entrad en la sala de chat del equipo al que quieréis enviar el mensaje y recupera su clave pública $(n_{destino}, e_{destino})$.
- A cada letra del mensaje, le asignaremos su número de carácter unicode. Para ello, podemos usar la función $ord(carácter)$ que recibe un carácter y devuelve su código unicode.
- Sean $m_1, \dots, m_k$ los números asociados a cada letra del mensaje. Para cada m_i calculamos

$$c_i = m_i^{e_{destino}} \pmod{n_{destino}}$$

- En la sala de chat del destinatario, uno de vosotros debe enviar el mensaje

“X: c_1 c_2 c_3 ...”

donde X es vuestra mesa y c_i son los caracteres cifrados que habéis obtenido. Por ejemplo, en la sala de C, el equipo D debería escribir algo similar a:

“D: 123513 104882 91032 151 0943”

7. Entrad en vuestra sala, leed el mensaje que habéis recibido y descifradlo letra a letra de forma similar al punto 5. Para obtener el carácter unicode con código m se puede usar la función $chr(m)$. Comprobad con vuestros compañeros de la mesa siguiente si el mensaje está correcto.
8. Intercambiad mensajes con otras mesas, enviando siempre los mensajes cifrados con la clave pública del destino y descifrando los que entren a vuestra sala usando vuestra clave privada.
9. Entrad a alguna de las salas de otra mesa y leed su clave pública (n, e) . Usando `modular.py/imatlab`, calculad la clave privada d de esa mesa (romped su clave RSA) y usadla para descifrar al menos un mensaje de su sala. Después, publicad dicho mensaje descifrado en su sala en texto plano.
10. En vuestra sala habréis recibido un mensaje de un usuario X, indicando su clave pública y un texto largo cifrado con dicha clave pública. Intentad descifrar dicho mensaje. ¿Qué escritora escribió el texto del usuario X?

FUNCIONES DE LA LIBRERÍA RSA.PY

Crea una librería llamada “rsa.py” e implementa las siguientes funciones

1. Implementar una función de generación de claves:

$$\text{generar_claves}(\text{min_primo} : \text{int}, \text{max_primo} : \text{int}) \rightarrow \text{Tuple}[\text{int}, \text{int}, \text{int}] :$$

que reciba dos enteros min_primo y max_primo , obtenga aleatoriamente dos primos p_1 y p_2 distintos en el intervalo $[\text{min_primo}, \text{max_primo})$ y, utilizándolos, calcule (n, e, d) con

$$n = p_1 p_2$$

$$ed \equiv 1 \pmod{\varphi(n)}$$

2. Implementar funciones de agregación y eliminación de relleno (*padding*). Utilizaremos el siguiente esquema de relleno. Fijamos un número de cifras para el relleno “*digitos_padding*”. Dado un mensaje m , la función

$$\text{aplicar_padding}(m : \text{int}, \text{digitos_padding} : \text{int}) \rightarrow \text{int}$$

devuelve el número cuyas primeras cifras son las de m , añadiendo al final *digitos_padding* cifras aleatorias adicionales.

La inversa de esta función es la función

$$\text{eliminar_padding}(m : \text{int}, \text{digitos_padding} : \text{int}) \rightarrow \text{int}$$

que, dado un entero m , elimina sus últimas *digitos_padding* cifras.

3. Implementar funciones de cifrado y descifrado de enteros usando RSA:

$$\text{cifrar_rsa}(m : \text{int}, n : \text{int}, e : \text{int}, \text{digitos_padding} : \text{int}) \rightarrow \text{int}$$

$$\text{descifrar_rsa}(c : \text{int}, n : \text{int}, d : \text{int}, \text{digitos_padding} : \text{int}) \rightarrow \text{int}$$

La función de cifrado toma un mensaje entero m y una clave pública (n, e) y devuelve el mensaje cifrado usando RSA con dicha clave y *digitos_padding* cifras de padding (ver (2)).

La función de descifrado toma un mensaje entero cifrado c y una clave privada (n, d) y devuelve el mensaje claro usando RSA con dicha clave y eliminando *digitos_padding* cifras de padding (ver (2)).

4. Usando las funciones anteriores, implementar funciones

$$\text{cifrar_cadena_rsa}(s : \text{str}, n : \text{int}, e : \text{int}, \text{digitos_padding} : \text{int}) \rightarrow \text{List}[\text{int}]$$

$$\text{descifrar_cadena_rsa}(cList : \text{List}[\text{int}], n : \text{int}, d : \text{int}, \text{digitos_padding} : \text{int}) \rightarrow \text{str}$$

que permitan cifrar y descifrar cadenas completas de texto usando RSA con claves (n, e) y (n, d) respectivamente y *digitos_padding* cifras de padding, según el esquema de relleno indicado anteriormente. Cada cadena de texto se identificará con la lista de enteros que represente los códigos unicode de cada uno de sus caracteres y el cifrado se realizará carácter a carácter. La función de cifrado tomará una cadena y devolverá la lista de enteros con los códigos cifrados. La función de descifrado toma como entrada la lista de enteros cifrados y devuelve la cadena descifrada completa como string.

5. Desarrollar e implementar un algoritmo

$$\text{romper_clave}(n : \text{int}, e : \text{int}) \rightarrow \text{int}$$

que calcule la clave privada d de un par de claves de RSA a partir de su clave pública (n, e) .

6. Investigar la noción de ataque de texto claro (elegido) e implementar una función:

$ataque_texto_elegido(cList : List[int], n : int, e : int) \rightarrow str$

que reciba como entrada una lista de enteros $cList$ que represente un texto s que ha sido cifrado carácter a carácter con RSA usando clave pública (n, e) sin padding y dicha clave pública (n, e) (es decir, $cList$ sería el resultado de hacer $cifrar_cadena_rsa(s, n, e, 0)$), ejecute un ataque de texto claro sobre ella y devuelva la cadena original s . Utilizar esta función para romper el criptograma recibido de “X” en el punto 10 de la práctica en grupo.

7. **(Opcional)** Implementa un algoritmo que permita atacar el sistema descrito en esta práctica y que pueda servir para romper el cifrado de un texto con claves generadas multiplicando dos primos de al menos 10 dígitos y al menos 2 cifras de padding.

ENTREGAS

Como resultado final de la práctica se entregará

- El código desarrollado, apropiadamente comentado y documentado (ver guía de estilo PEP8 en Moodle):
 - La librería “modular.py”, actualizada y corregida según sea necesario.
 - La librería “rsa.py”.
 - El programa “registrarusuario.py”
 - El programa “criptochar.py”.
 - Cualquier otra librería, script o fichero de configuración adicional que haya sido programada por los alumnos para el desarrollo de la práctica y que sea necesario para ejecutar los scripts.
- Una memoria en PDF con el nombre P2GPxxx.pdf (donde GPxxx es el código de grupo de la pareja) en el que se especificarán, como mínimo, los siguientes elementos:
 - Nombre y apellidos de los integrantes del grupo y código del grupo de prácticas (GPxxx).
 - Descripción de los cambios o mejoras implementadas en “modular.py” respecto a la práctica 1.
 - Descripción del modo en el que se ha implementado la generación de claves. ¿Es capaz el código de generar claves grandes (más de 20 dígitos) en un tiempo razonable?
 - Descripción y justificación del modo en que se realizan los ataques a RSA en las funciones “romper_clave” y “ataque_texto_elegido”. ¿Pueden estas funciones romper cualquier tipo de cifrado con el esquema establecido en la práctica o hay alguna limitación?
 - Texto claro recuperado a partir del texto cifrado recibido en la práctica presencial.
 - Si se ha implementado, descripción y justificación de la estrategia de ataque optativa implementada.
 - Cualquier comentario que se considere relevante sobre el diseño de la implementación.
 - Bibliografía, incluyendo referencias debidamente citadas a cualquier fuente utilizada para desarrollar la práctica.

Todos los ficheros anteriores se cargarán en la tarea correspondiente de Moodle. Salvo que sea imprescindible, **los ficheros deben subirse individualmente a Moodle, no en una carpeta comprimida**. Los programas “registrarusuario.py” y “criptochar.py” deberán poder ejecutarse en los laboratorios sin errores.

CRITERIOS DE EVALUACIÓN

- **Librería “modular.py” (1.5 puntos):** Capacidad de la librería (actualizada según sea necesario) de realizar las tareas necesarias para que se puedan ejecutar los nuevos módulos de forma correcta y eficiente.
- **Diseño general del programa “criptochat.py” (1 punto):** Se puntuará el diseño general del programa, la implementación correcta de la entrada/salida y la gestión correcta de los casos y mensajes de error. El programa deberá ejecutarse sin bugs y no deberá entrar en un bucle infinito o cerrarse inesperadamente, independientemente de la entrada introducida por el usuario.
- **Implementación de las funciones solicitadas en “rsa.py” (6 puntos):** Se sumará la puntuación de cada una de las funciones implementadas (máximo 1 punto por problema si las funciones se ejecutan de forma correcta y con una eficiencia razonable). En el caso de los ataques a RSA, se tendrá en cuenta si se ha descifrado correctamente el cifrado pedido. Se puntuará positivamente la implementación del ataque a RSA opcional (hasta +1 punto).
- **Organización y calidad del código (0.5 puntos):** Se valorará el uso de una programación estructurada, siguiendo un buen estilo de programación, la modularidad y reutilización del código, así como que el código esté correctamente documentado.
- **Memoria (1 punto):** Se valorará que la descripción de los algoritmos pedidos sea correcta y completa.

IMPORTANTE

Es obligatorio preservar las interfaces de los módulos proporcionados (es decir, los tipos de datos que reciben y devuelven las funciones dadas en la práctica no pueden ser modificados). Por ejemplo, si una función devuelve un entero, en ningún caso puede devolver un String. Es así mismo obligatorio que cada grupo pruebe, al menos:

- Que su código se ejecuta adecuadamente en los laboratorios de ICAI mediante una llamada desde la terminal.
- Que los tests básicos de pytest suministrados como material en las prácticas se ejecutan correctamente para sus funciones.

No realizar estas pruebas mínimas de funcionamiento antes de entregar supondrá un suspenso en la práctica.